

Systems and Methods for Big and Unstructured Data - Notes - v0.1.0

260236

September 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

 [GitHub repository](#)



These notes are an unofficial resource and shouldn't replace the course material or any other book on systems and methods for big and unstructured data. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1	Big Data	4
1.1	The Big Data and AI Market Landscape	4
1.2	Technology evolution and the Hype Cycle	9
	Index	14

1 Big Data

1.1 The Big Data and AI Market Landscape

The first idea to establish is that **Big Data is not a single technology**. When people say “*Big Data*”, they may be referring to databases, distributed storage, analytics platforms, cloud infrastructure, machine learning, visualization tools, or complete applications. In reality, these components form an **ecosystem**. A useful simplified picture is:

Data Sources → Data Infrastructure → Analytics → Applications

Different companies and technologies operate at different points of this pipeline.

≡ Data sources: where everything starts

Before we can have “Big Data”, something must generate data. Modern systems continuously produce information from many heterogeneous sources like web, smartphones, IoT devices, social networks, transactions, sensors, and more.

Example 1: E-Commerce data sources

For example, imagine an e-commerce service. It might collect: customer profiles, purchases, searches, clicks, page views, product reviews images, payment transactions, delivery information. Notice immediately that **these are not all the same kind of data**. Some may be highly structured:

```
1 order_id = 58321
2 customer_id = 912
3 price = 49.99
```

While others may be much less structured:

“This product arrived late but works very well.”

Or could be images, video, GPS coordinates, logs, etc. This heterogeneity will later become one of the defining properties of Big Data: **Variety**.

≡ The infrastructure layer

Once data exists, we need technologies capable of **storing, transporting and processing it**. This is the infrastructure part of the Big Data market. Conceptually:

Raw Data → **Infrastructure** → usable data

Infrastructure contains things such as: distributed storage; databases; distributed computing systems; data ingestion systems; messaging systems; cloud infrastructure. Later in the notes we will encounter technologies belonging to this general world, such as distributed databases and large-scale processing architectures.

The fundamental challenge is that a **traditional architecture** such as *one database on one machine* may eventually become **insufficient**. Then the architecture can evolve toward something like:

Machine 1	Machine 2	Machine 3
data	data	data

But in the other hand, a **distributed architecture** introduces new questions about partitioning, replication, consistency, failures, and scalability. These questions will be central later in the notes.

🔗 Open-source platforms

Another important part of the Big Data ecosystem is the strong presence of **open-source platforms**. Historically, much of the Big Data revolution was enabled by systems that made distributed storage and computation available without requiring every company to implement these mechanisms from scratch. Instead of an organization implementing distributed file storage, fault recovery, parallel computation, cluster scheduling, data partitioning from zero, it can **build on an existing platform**.

So we can think of these platforms as **providing a reusable technological foundation**:

Distributed infrastructure → platform → data applications

A famous historical example that we will study later is the *Hadoop* ecosystem.

📊 Analytics layer

Simply storing huge quantities of data does not create value. Imagine a company has 100 TB of customer data. If nobody can derive useful information from it, those 100 TB are mostly a **cost**.

We therefore need another layer:

Stored Data → **Analytics** → Information / Models / Predictions

Analytics can include things such as: statistics; aggregation; data mining; machine learning; recommendation systems; forecasting; anomaly detection.

🔗 Example 2: Netflix analytics

Suppose Netflix records:

- User A watched movie X
- User A paused at 32 minutes
- User A finished movie Y
- User A abandoned movie Z

The **infrastructure problem** is: “*Can we reliably collect and store billions of these events?*”. The **analytics problem** is: “*Can we infer something useful from them?*”. For instance:

$$P(\text{user likes movie } m \mid \text{previous behaviour})$$

That distinction between **managing data** and **extracting information from data** is important.

Application layer

At the top, analytics becomes something the user or organization can actually use. For example:

behaviour data → recommendation model → “Recommended for you”

The user does not see the distributed storage system, cluster, database, or machine-learning infrastructure. They see the **application**. Other examples might include:

traffic data → traffic prediction → route recommendation

Or:

financial transactions → fraud detection model → fraud alert

Or:

sensor readings → predictive model → maintenance warning.

So there is a **transformation**:

raw data → processed data → insight → action

Full data-stack providers

Not every company operates at only one layer. Some vendors provide an almost complete data stack. Instead of offering only a database, or only analytics, they might offer:

storage + processing + analytics + ML + visualization + application services

These are the **full data stack providers**.

Example 3

Cloud platforms are an intuitive modern example of this idea.

A single provider may offer services for: object storage, databases, stream processing, distributed computing, data warehouses, ML training, model deployment, and business intelligence.

Therefore, the borders between the individual categories of the market are not strict.

🌱 Where does AI fit?

The AI market is strongly connected to this Big Data ecosystem because modern AI depends heavily on data infrastructure. A simplified relationship is:

Data → Data Engineering → Machine Learning / AI → Application

For example, suppose we want to train a recommendation model. The ML algorithm itself might be only one piece. Before training it, somebody must:

1. Collect user interactions;
2. Clean them;
3. Integrate different sources;
4. Store them;
5. Transform them into suitable features;
6. Feed them efficiently to the model.

Thus **AI requires a data system underneath it**. A powerful model with poor data infrastructure is often practically useless.

🔪 **Big Data and AI are therefore not synonyms.** This distinction is useful.

- **Big Data** is primarily concerned with questions such as: “*How do we collect, store, integrate, manage and process huge or complex datasets?*”
- **AI/ML** is more concerned with: “*How do we learn patterns or decision functions from data?*”

There is obviously a **large overlap** because **AI systems often consume enormous quantities of data**. But they solve different categories of problems. For Systems and Methods for Big and Unstructured Data (SMBUD), our main perspective is the **data-management side**. For example, before asking “*which neural network should we use?*”, we are much more likely to ask:

- *Where does the data live?*
- *How is it represented?*
- *How is it partitioned?*
- *How do we query it?*
- *Can the system scale?*
- *What consistency guarantees do we have?*

📌 Example 4: Spotify recommendation pipeline

Imagine Spotify wants to recommend songs. At the bottom we have **data sources**: play events, skips, likes, searches, playlists, artists, songs,

devices, locations. These might generate billions of events. Then:

1. **Data Sources** (user events, song metadata, etc.) generate data.
2. **Infrastructure** collects and stores them: Distributed Storage, Databases, Streaming Systems.
3. **Analytics layer** looks for patterns: $P(\text{user } u \text{ likes song } s)$.
4. **Application** shows: Discover Weekly, Recommended songs, Similar artists.

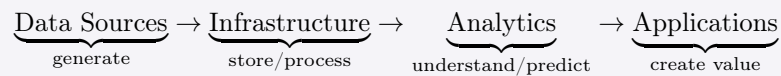
Therefore:

User events $\rightarrow$ Big Data infrastructure $\rightarrow$ Analytics/AI $\rightarrow$ Product feat

This is essentially the market landscape represented as a technical pipeline.

✓ Key Takeaways 1

We don't need to memorize the Big Data market as a collection of company logos. What matters is recognizing the **roles** inside the ecosystem:



With **open-source platforms** providing many of the underlying technological building blocks and **full-stack providers** potentially spanning several or all of these layers.

And the AI market sits naturally on top of and alongside this ecosystem because **without a scalable data pipeline, scalable AI is extremely difficult.**

1.2 Technology evolution and the Hype Cycle

When a new technology appears, its adoption is rarely linear. A common pattern is:

innovation → hype → disappointment → maturity

This is what the **Hype Cycle** tries to describe. The important point is that the **perceived importance of a technology and its real practical maturity are not the same thing.**

A technology can be talked about everywhere, heavily funded, considered revolutionary, while still being technically immature or poorly understood.

📐 The basic shape

The Hype Cycle is usually represented as a curve with:

$x = \text{time}, \quad y = \text{expectations}$

The typical progression is:

1. **Technology Trigger:** a new technology appears, with prototypes and promising research results.
2. **Peak of Inflated Expectations:** enthusiasm grows, and people start expecting the technology to solve almost everything.
3. **Trough of Disillusionment:** reality catches up with expectations, and limitations become visible.
4. **Slope of Enlightenment:** people start understanding what the technology is actually good for, and real use cases emerge.
5. **Plateau of Productivity:** the technology becomes mature, and it reliably solves a known class of problems.

Let's understand each phase.

⚡ 1. Technology Trigger

A new technology appears. At this point:

- There may be prototypes;
- Research results are promising;
- Companies start experimenting;
- Practical use cases are still limited.

The technology is interesting, but not yet mature. For Big Data, this kind of **phase happened when ideas** such as large-scale distributed data processing and new NoSQL systems **began gaining attention.** The important thing is that the **potential is high, but evidence is still limited.**

🔗 2. Peak of Inflated Expectations

Then enthusiasm grows. People start expecting the technology to solve almost everything. We often hear claims such as: “*this will completely replace existing systems*” or “*every company needs this*” or “*this changes everything*”. Some early projects succeed, but **many expectations are unrealistic**. This is the **hype** phase.

For example, imagine a new database technology is introduced. Initially people may start saying: “*relation databases are obsolete*” or “*every dataset should be moved to this new system*”. That is usually an **overreaction**. A new technology may be excellent for certain workloads without being universally superior.

And this point is especially relevant in SMBUD. Later, when we see MongoDB, Cassandra, Redis, Neo4j, Elasticsearch, vector databases, we should **not** think:

“newer technology” = “better technology”

Instead: **technology suitability depends on requirements**. For instance, a graph database is not “*better*” than a relational database in general. It may be much better for relationship-heavy queries, while a relational DBMS may be more appropriate for another application.

The Hype Cycle is therefore also a warning against **technology-driven design**. We should not choose a technology because it is fashionable.

🔗 3. Trough of Disillusionment

Eventually, **reality catches up with expectations**. Projects fail. Limitations become visible. People discover problems such as: high operational complexity; scalability limits; poor tooling; expensive maintenance; missing features; difficult migration; lack of expertise.

Enthusiasm then collapses. The technology may suddenly look much less revolutionary. This is the **Trough of Disillusionment**. But this does **not necessarily mean that the technology is bad**. Instead, unrealistic expectations are being corrected.

💡 4. Slope of Enlightenment

At this stage, **people start understanding**: “*What is this technology actually good for?*”. Instead of saying: “*use it everywhere*”. The discussion becomes: “*use it when requirements X, Y, Z are present*”. Real use cases emerge. Best practices become clearer. Tooling improves. Companies gain operational experience. This is **when the technology starts becoming genuinely useful**.

✅ 5. Plateau of Productivity

Eventually, the **technology becomes mature**. It is no longer interesting because it is fashionable. It is **interesting because it reliably solves a known class**

of problems. At this point: use cases are understood; benefits are measurable; limitations are known; tooling is mature; engineers know how to operate it. The technology has moved from *hype* to *engineering*. And that distinction is very important.

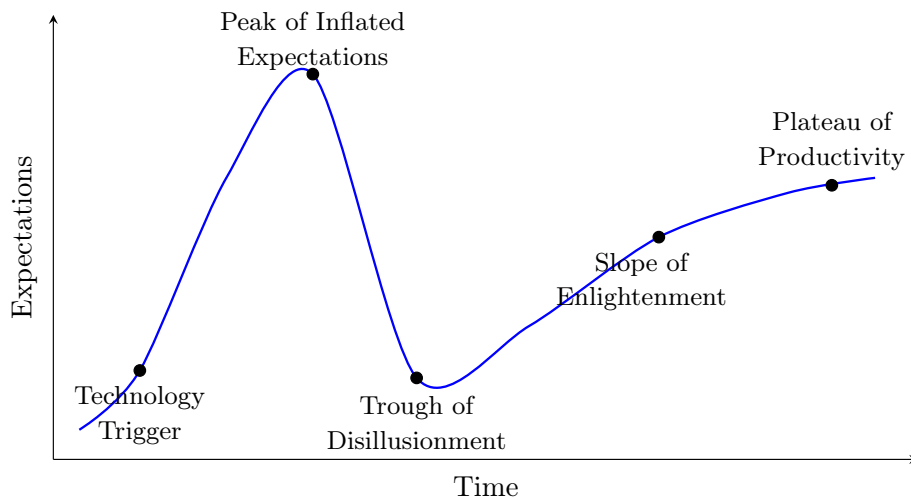


Figure 1: **The Hype Cycle.** The curve represents the evolution of expectations over time for a new technology.

Example 5: NoSQL

Consider how people may react when NoSQL systems appear. At the beginning:

“Interesting alternative to relational databases”

Then hype grows:

“SQL is dead!”

Eventually problems emerge:

- limited joins
- weaker consistency in some systems
- application complexity
- data duplication
- operational challenges

So enthusiasm decreases. Later, the industry develops a more nuanced understanding:

- Cassandra is useful for certain distributed, write-heavy workloads.

- Neo4j is useful when relationships and graph traversals dominate.
- MongoDB is useful when document-oriented structures align well with the application.
- PostgreSQL remains excellent for many transactional systems.

This mature view is exactly what we want in SMBUD.

📌 The connection with the Big Data market

The reason the Hype Cycle appears right after the Big Data and AI market landscape is that **markets around emerging technologies are extremely dynamic**. When a technological wave appears:

new problem → new technologies → new companies → huge expectations

Many vendors enter the market. But some disappear, some merge, some technologies become standard, others remain niche, and others fail entirely. So the market landscape we saw earlier should not be interpreted as a fixed map. It **changes continuously as technologies mature**.

❓ Why this matters for a data engineer

Imagine that our company says: “*Everyone is using vector databases. We should migrate everything to a vector database*”. A good engineer should **not answer**: “*Yes, because vector databases are modern*”. Instead, we ask:

- What are our queries?
- What type of data do we have?
- What latency do we need?
- What consistency guarantees?
- What scale?
- What problem are we solving?

Then we evaluate whether the technology fits those requirements. That is the engineering mindset the Hype Cycle encourages us to adopt.

📌 Key Takeaways 2

The **Hype Cycle** is essentially a model of the gap between *expectations* and *actual technological maturity*. The typical evolution is: (1) Trigger, (2) Inflated Expectations, (3) Disillusionment, (4) Understanding, (5) Productive Use.

For us, the practical lesson is that **we should not choose a technology because it is fashionable**. Instead, we should choose it because its **data model, architecture, and trade-offs fit the problem we are trying to solve**.

References

- [1] Brambilla Marco. Systems and methods for big and unstructured data. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

H

Hype Cycle

9